Genetic Algorithms (GA)

IWRI Workshop day 2

Dr. Stefaan Haspeslagh

Vives & KU Leuven KULAK

April 8, 2026

Agenda

1. From Single-Solution to Population Search
2. The Three GA Operators
3. The Canonical GA
4. Memetic GA: Combining GA and Local Search
5. TSP Case Study

Roadmap

1. From Single-Solution to Population Search
2. The Three GA Operators
3. The Canonical GA
4. Memetic GA: Combining GA and Local Search
5. TSP Case Study

The problem with single-solution methods

Both local search and simulated annealing maintain and evolve **one** current solution:

- LS: moves greedily to a better neighbour; terminates at a local optimum.
- SA: escapes local optima stochastically via the Metropolis criterion.

Limitations of a single trajectory:

- One trajectory samples one path through the solution space.
- No mechanism to recombine "good parts" from different solutions.
- Multi-start helps, but restarts are fully *independent* — no information sharing.

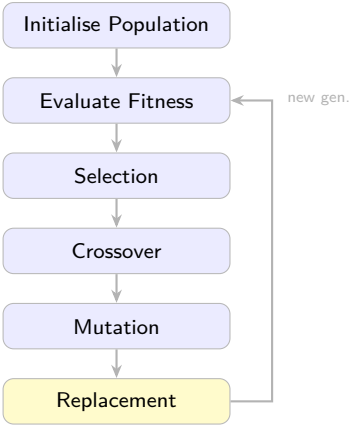
*What if we could evolve many solutions simultaneously, and **combine** their best features?*

The Genetic Algorithm idea

Nature's metaphor

- **Individual** ↔ candidate solution
- **Chromosome** ↔ encoding of solution
- **Gene** ↔ single decision variable
- **Fitness** ↔ solution quality
- **Selection** ↔ survival of fittest
- **Crossover** ↔ sexual recombination
- **Mutation** ↔ random gene change
- **Generation** ↔ one iteration

GA Cycle



Why populations help: building blocks

Schema Theorem (Holland 1975). Short, low-order schemata (patterns that appear in many good solutions) receive exponentially increasing representation over generations — the "building blocks hypothesis".

Intuition for TSP. Suppose two tours both contain the excellent sub-sequence $\pi_i \to \pi_{i+1} \to \pi_{i+2}$ (a locally optimal chain of three cities). Crossover can preserve this chain in offspring while exploring different orderings of the remaining cities.

What a GA buys over multi-start SA:

- **Diversity**: the population simultaneously explores multiple basins of attraction.
- **Recombination**: offspring inherit good sub-structures from both parents, not just a mutated copy of one.
- **Implicit parallelism**: N individuals effectively evaluate many schemata per generation.

Roadmap

1. From Single-Solution to Population Search
2. The Three GA Operators
3. The Canonical GA
4. Memetic GA: Combining GA and Local Search
5. TSP Case Study

Initialisation

The initial population should be **diverse** (to explore broadly) but not entirely random (so the GA does not waste generations on obviously bad solutions).

Pure random initialisation. Generate N random permutations. Simple, maximum diversity, but all tours are very long initially.

Mixed initialisation (Prins 2004). *Recommended for TSP.*

- Seed $\approx 15\%$ of the population with nearest-neighbour (NN) tours, each starting from a different city.
- Fill the remaining 85% with random permutations.

Why? The NN tours provide a "quality floor" that the GA can immediately improve via crossover. The random majority provides diversity so the GA does not converge prematurely.

Rule of thumb. Avoid starting with too many NN tours: if the whole population is NN-seeded, diversity collapses immediately and the GA behaves like multi-start LS.

Selection: tournament selection

Selection decides which individuals reproduce. We need **selection pressure** (better individuals reproduce more often) without **premature convergence** (only the very best individual dominate).

Tournament selection (recommended):

1. Draw k individuals uniformly at random from the population (with replacement).
2. The individual with the lowest cost (best fitness) among the k candidates wins.
3. Return a copy of the winner as a parent.

Tournament size k	Selection pressure	Effect
2	Low	Preserves diversity; slow convergence
3–5	Medium	Good default; balances exploration/exploitation
$\gg 5$	High	Converges quickly; risk of premature convergence

Tournament selection — numeric example

Population of N=6 tours with costs below. We run two tournaments, with k=3, to select two parents.

Tournament 1 (k=3): draw π_2, π_4, π_5 winner $\rightarrow p_1$

π_1	π_2	π_3	π_4	π_5	π_6
480	510	495	530	**470**	505

Tournament 2 (k=3): draw $\pi_1, \pi_3, \pi_6 \rightarrow$ winner π_1 (480) $\rightarrow p_2$

$p_1 = \pi_5$ (cost 470), $\quad p_2 = \pi_1$ (cost 480) $\Rightarrow$ these become the crossover parents.

Note. Tournament is *rank-based*: only the relative ordering within each draw matters, not the absolute cost values. Compare this with fitness-proportionate selection on the next slide.

Two selection paradigms in GAs

GAs need to give better individuals a higher reproduction chance. Two classical approaches:

Fitness-proportionate ("roulette wheel")
Holland 1975

1. **Fitness scaling**: convert cost to positive fitness, e.g. $f_i = C_{\max} - \text{cost}_i$
2. Prob. $p_i = f_i / \sum_j f_j$
3. Sample proportionally

Pressure depends on *absolute values* of f_i.
Sensitive to outliers and to convergence.

Tournament selection *(recommended)*
De Jong 1975

1. Draw k individuals at random
2. Winner = lowest cost among the k
3. No scaling, no probability formula

Pressure depends on *rank within the draw*.
k is the single tuning parameter.

Property	Fitness-proportionate	Tournament
Fitness scaling required	Yes	No
Outlier sensitivity	High	Low (rank-based)
When population converges	Pressure collapses	Stable via k

Fitness scaling — numerical illustration

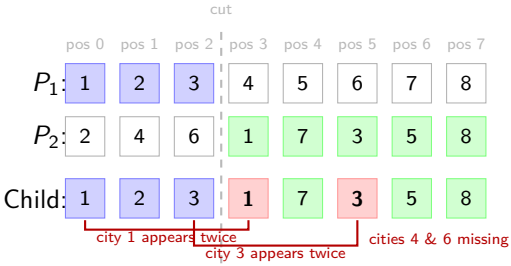
Fitness scaling: $f_i = C_{\max} - \text{cost}_i$ converts cost (minimise) to fitness (maximise). Same population as slide 11 ($C_{\max} = 530$):

			Fitness-proportionate		Tournament (k=3)	
Ind.	Cost	Rank	f_i	p_i	T1: π_2, π_4, π_5	T2: π_1, π_3, π_6
π_1	480	2nd	50	26.3%	—	**winner**
π_2	510	5th	20	10.5%	drawn	—
π_3	495	3rd	35	18.4%	—	drawn
π_4	530	last	**0**	**0%**	drawn	—
π_5	470	1st	60	31.6%	**winner**	—
π_6	505	4th	25	13.2%	—	drawn

- **Outlier kills diversity.** π_4 has $p_4 = 0\%$: it can *never* be selected under roulette, even though it can still be drawn in a tournament.
- **Formula dependency.** Using $f_i = 1/\text{cost}_i$ instead changes every p_i; selection pressure depends entirely on the chosen scaling formula.
- **Late-gen. collapse.** When costs converge, all f_i become nearly equal $\Rightarrow p_i \to 1/N \Rightarrow$ near-random selection. Tournament holds pressure via k.

Why standard crossover fails for TSP tours

In a TSP chromosome, **gene i = the city visited at step i**. Because every city must be visited exactly once, a chromosome is a *permutation* of city labels — each label appears exactly once. Naively splicing two parents at a cut point destroys this property.



Conclusion. Any TSP crossover operator must guarantee each city appears exactly once in the child. Two operators do this: **OX** and **PMX** (next slides).

Order Crossover (OX) — Davis (1985)

What OX preserves: the *relative order* of cities from P_2 (key for Euclidean TSP).

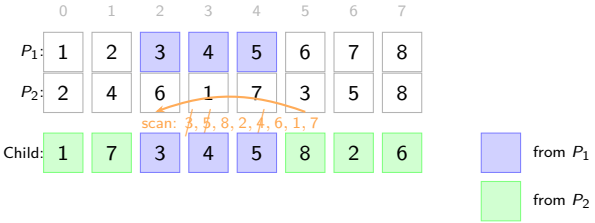
Algorithm. (1) Copy segment $P_1[a : b{+}1]$ into the child. (2) Scan P_2 from $b{+}1$ (wrapping); fill remaining child positions in the same order, skipping cities already in the segment.

Example: $P_1 = (1\ 2\ \mathbf{3\ 4\ 5}\ 6\ 7\ 8)$, $P_2 = (2\ 4\ 6\ 1\ 7\ 3\ 5\ 8)$, cut $[a{=}2,\ b{=}4]$

	0	1	2	3	4	5	6	7	
P_1:	1	2	3	4	5	6	7	8	
P_2:	2	4	6	1	7	3	5	8	← step 2 scans from here
Child:	–	–	3	4	5	–	–	–	step 1: copy segment

OX — completing the child

Scan P_2 from pos 5 (wrap): $\cancel{3}$, $\cancel{5}$, 8, 2, $\cancel{4}$, 6, 1, 7. Keep $\notin \{3,4,5\}$: $\mathbf{8}, \mathbf{2}, \mathbf{6}, \mathbf{1}, \mathbf{7}$ (fill positions $5 \to 6 \to 7 \to 0 \to 1$ in wrap order).



Child $= (1\ 7\ \mathbf{3}\ \mathbf{4}\ \mathbf{5}\ 8\ 2\ 6)$ ✓ Cities $8, 2, 6, 1, 7$ appear in the *same relative cyclic order* as in P_2.

Partially Mapped Crossover (PMX) — Goldberg & Lingle (1985)

What PMX preserves: the *absolute positions* of cities in the copied segment (from P_2) and outside it (from P_1, with conflict resolution).

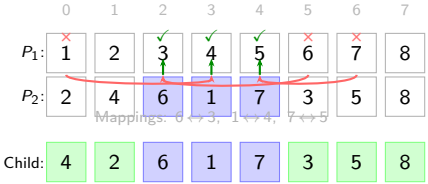
Algorithm:

1. Pick cut points $a \leq b$; copy $P_2[a : b{+}1]$ into the child.
2. For each position k *outside* $[a, b]$: try to place $P_1[k]$.
   If $P_1[k]$ is already in the segment $\Rightarrow$ follow the **mapping chain**: find where $P_1[k]$ sits in P_2 (say position m), then try $P_1[m]$ instead. Repeat until a non-conflicting city is found.

Same example. $P_1 = (1\ 2\ 3\ 4\ 5\ 6\ 7\ 8)$, $P_2 = (2\ 4\ 6\ 1\ 7\ 3\ 5\ 8)$, cut $[2, 4]$:
Segment from P_2: $\{6, 1, 7\}$ at positions $[2, 3, 4]$. Mapping: $6 \leftrightarrow 3$, $1 \leftrightarrow 4$, $7 \leftrightarrow 5$.

PMX — resolving conflicts via mapping chains

Segment $\{6, 1, 7\}$ at positions $[2, 3, 4]$. For each position outside $[2, 4]$: place $P_1[k]$, or follow the chain if conflict.



Child $= (4\ 2\ \mathbf{6}\ \mathbf{1}\ \mathbf{7}\ 3\ 5\ 8)$ Valid? ✓

OX vs PMX — which to prefer?

Property	OX	PMX
What it preserves	Relative order (from P_2)	Absolute positions (P_2 in segment; P_1 outside)
Complexity	$O(n)$	$O(n)$
Best suited for	Euclidean TSP	Problems where position matters
Handling of wrap	Wrap-around scan of P_2	Mapping chain (no wrap needed)

Practical recommendation.

- For Euclidean TSP: **OX** is the more widely reported choice (Larrañaga et al. 1999).
- Both operators produce similar results in practice; OX is slightly simpler to implement.
- Both outperform standard one-point crossover on TSP by a wide margin.

Mutation operators

Mutation maintains genetic diversity by introducing small random perturbations. Without mutation, the GA can only recombine existing material — it cannot escape to regions not represented in the initial population.

Operator	Operation	Cost	Connection to LS	Recommendation
Inversion	Reverse segment $[\pi_i, \ldots, \pi_j]$	$O(n)$	$\equiv$ one 2-opt move	**Preferred**
Swap	Exchange $\pi_i \leftrightarrow \pi_j$	$O(1)$	Not a 2-opt move	Simple backup

Rate recommendation. $p_m = 1/n$ per individual: on average one mutation event per tour per generation.

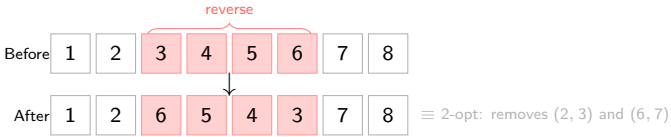
- Too high ($p_m \gg 1/n$): destroys good offspring, random walk.
- Too low ($p_m \ll 1/n$): premature convergence, reduced diversity.

Why inversion? It is identical to a 2-opt neighbourhood move, so it can perform the same quality improvements that LS would — a natural bridge between the GA and local search.

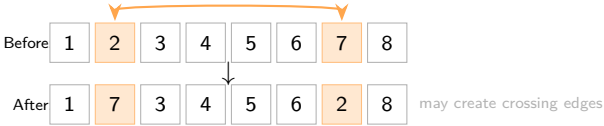
Mutation — step-by-step examples

Starting tour: $\pi = (1\ 2\ 3\ 4\ 5\ 6\ 7\ 8)$

Inversion mutation at i=2, j=5: reverse segment $[\pi_2, \ldots, \pi_5]$.



Swap mutation at i=1, j=6: exchange $\pi_1 \leftrightarrow \pi_6$.



Roadmap

1. From Single-Solution to Population Search
2. The Three GA Operators
3. The Canonical GA
4. Memetic GA: Combining GA and Local Search
5. TSP Case Study

Canonical GA pseudocode (1/2)

```
def ga_tsp(cost, N=80, G=200, pc=0.90, pm=None,
           k_tournament=3, elites=1,
           mixed_init_frac=0.15, crossover="ox"):
    """
    cost           : n x n distance matrix
    N              : population size
    G              : number of generations
    pc             : crossover probability
    pm             : mutation probability (default 1/n)
    k_tournament   : tournament size
    elites         : number of elite individuals copied unchanged
    mixed_init_frac: fraction seeded with nearest-neighbour tours
    """
    if pm is None:
        pm = 1.0 / n

    # -- Initialise population (mixed: NN + random) --
    n_greedy = round(N * mixed_init_frac)
    P = [nn_tour(cost, start=r) for r in range(n_greedy)]
    P += [random_permutation(n) for _ in range(N - n_greedy)]
    fitness = [tour_length(cost, x) for x in P]
```

Canonical GA pseudocode (2/2)

```
    best = min(P, key=lambda x: tour_length(cost, x))

    for g in range(G):
        # Elitism: copy e best individuals to next generation
        sorted_P = sorted(P, key=lambda x: tour_length(cost, x))
        new_P = sorted_P[:elites]

        # Fill rest of new population
        while len(new_P) < N:
            p1 = tournament_select(P, fitness, k_tournament)
            p2 = tournament_select(P, fitness, k_tournament)
            if random() < pc:
                c1, c2 = crossover_ox(p1, p2), crossover_ox(p2, p1)
            else:
                c1, c2 = p1[:], p2[:]           # clone
            mutate_inversion(c1, pm)
            mutate_inversion(c2, pm)
            new_P.extend([c1, c2])

        P = new_P[:N]
        fitness = [tour_length(cost, x) for x in P]
        best = min(best, min(P, key=...), key=tour_length)

    return best
```

Elitism

Problem without elitism. Crossover and mutation are stochastic: the best individual in generation g may not survive to generation $g+1$. Without a preservation mechanism, the best solution found can be lost.

Elitism. Copy the e best individuals from the current population unchanged into the next generation. This guarantees the best solution is monotonically non-increasing over generations.

Elites e	Effect	Recommendation
0	No preservation; best can be lost	Avoid
1	Preserve single best; maximum selection pressure	**Typical default**
2–3	Preserve a small elite group	Good for noisy fitness
$\gg 3$	Reduces effective population size; premature convergence	Avoid

Note. With $e = 1$ and $N = 80$, the elite occupies just 1.25% of the population slot — negligible cost, significant benefit.

Key parameters and typical values

Parameter	Symbol	Typical range	Effect
Population size	N	50–200; $\approx n$	Diversity vs. cost per generation
Generations	G	100–500	Total evolutionary budget
Crossover rate	p_c	0.85–0.95	Recombination frequency
Mutation rate	p_m	$1/n$ to 0.05	Exploration intensity
Tournament size	k	2–5	Selection pressure
Elites	e	1–3	Preservation of best
Greedy fraction	f_g	0.10–0.20	Quality floor in initial pop.

Tuning order of priority:

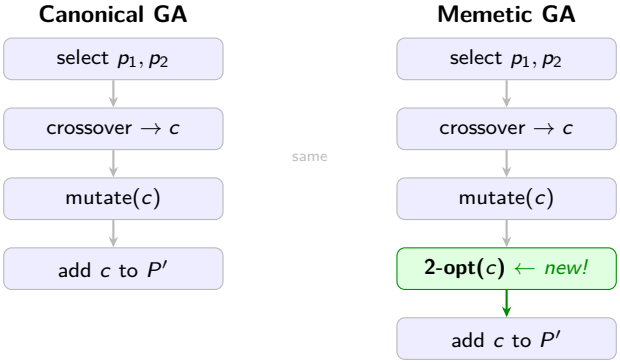
1. N and G: fix the total budget $N \times G$ first.
2. p_c and p_m: use defaults (0.90, $1/n$) unless performance is poor.
3. k: increase if convergence is too slow; decrease if diversity collapses early.

Roadmap

1. From Single-Solution to Population Search
2. The Three GA Operators
3. The Canonical GA
4. Memetic GA: Combining GA and Local Search
5. TSP Case Study

The Memetic GA idea

Observation. Crossover produces offspring with good *global* structure (right city order) but occasional *local* defects at splice points (crossing edges). A quick 2-opt pass removes these. **Memetic GA** (Moscato 1989; Mühlenbein 1992): apply **local search to every offspring**.



Result: the entire population consists of 2-opt local optima. Crossover recombines these, then LS re-polishes — dramatically improving solution quality.

Memetic GA pseudocode (change from canonical GA)

```
def memetic_ga_tsp(cost, N=50, G=100, pc=0.90, pm=None,
                   k_tournament=3, elites=1,
                   mixed_init_frac=0.15, ls_passes=3):
    """
    ls_passes : number of first-improvement 2-opt passes
                applied to each offspring (1 = one sweep;
                high = full local optimum)
    All other parameters as in ga_tsp.
    """
    # -- Initialise + 2-opt initial population --
    P = [two_opt_polish(t, ls_passes) for t in init_population()]

    for g in range(G):
        ...  # same as canonical GA until offspring creation:
        c1, c2 = crossover_ox(p1, p2), crossover_ox(p2, p1)
        mutate_inversion(c1, pm)
        mutate_inversion(c2, pm)
        c1 = two_opt_polish(c1, ls_passes)   # <-- key change
        c2 = two_opt_polish(c2, ls_passes)   # <-- key change
        new_P.extend([c1, c2])
    return best
```

Roadmap

1. From Single-Solution to Population Search
2. The Three GA Operators
3. The Canonical GA
4. Memetic GA: Combining GA and Local Search
5. TSP Case Study

GA on TSP: what the algorithm "sees"

Recall the representation: a tour is a permutation $\pi = (\pi_0, \pi_1, \ldots, \pi_{n-1})$. Two tours can share a sub-sequence (building block) even if their other cities differ.

Example with $n = 8$ cities.

$$P_1 = (3\ 1\ \mathbf{5}\ \mathbf{7}\ \mathbf{2}\ 4\ 8\ 6), \quad P_2 = (7\ 3\ \mathbf{5}\ \mathbf{7}\ \mathbf{2}\ 1\ 6\ 8)$$

Wait — P_2 already contains a duplicate! This is why parents must be produced by valid crossover, not arbitrary splicing.

Correct scenario. Both P_1 and P_2 happen to visit cities 5, 7, 2 consecutively (though at different positions). OX crossover can potentially transfer this chain into the offspring, preserving a good sub-tour.

One complete GA generation — trace

Setting. $n = 8$, $N = 6$, $k = 2$, $p_c = 1$, $p_m = 1/8$.

Step 1 — Population with fitness:

#	Tour	Cost
1	(1 2 3 4 5 6 7 8)	412
2	(2 4 6 1 7 3 5 8)	398
3	(5 1 3 7 2 8 4 6)	435
4	(8 3 6 2 4 7 1 5)	421
5	(3 7 1 5 8 2 6 4)	**387**
6	(6 2 8 4 3 5 1 7)	409

Step 2 — Elitism. Individual 5 (cost 387) is copied to next generation unchanged.

One complete GA generation — trace (cont.)

Step 3 — Tournament selection ($k = 2$):

Tournament A: draw $\{1\ (412),\ 3\ (435)\} \Rightarrow$ winner: individual 1 $\quad (p_1)$
Tournament B: draw $\{2\ (398),\ 6\ (409)\} \Rightarrow$ winner: individual 2 $\quad (p_2)$

$p_1 = (1\ 2\ \mathbf{3\ 4\ 5}\ 6\ 7\ 8), \quad p_2 = (2\ 4\ 6\ 1\ 7\ 3\ 5\ 8), \quad \text{cut } [2, 4].$

Step 4 — OX crossover:
Segment from p_1: cities $\{3, 4, 5\}$ at positions $[2, 3, 4]$.
Scan p_2 from pos 5: $3^\star, 5^\star, 8, 2, 4^\star, 6, 1, 7 \rightarrow$ keep $\{8, 2, 6, 1, 7\}$.

Child: positions $[5, 6, 7, 0, 1] \leftarrow [8, 2, 6, 1, 7]$:

$$c_1 = (1\ 7\ \mathbf{3\ 4\ 5}\ 8\ 2\ 6) \quad \text{cost: } 401$$

Step 5 — Inversion mutation ($p_m = 1/8$): no mutation this time (random $> 1/8$).
Population is updated; repeat selection/crossover until N individuals filled.

GA convergence behaviour

What to expect over generations:

- **Early generations (exploitation of initial diversity).** Good building blocks spread rapidly through the population via crossover; fitness improves quickly.
- **Middle generations.** Improvement slows as the population begins to converge; crossover recombines increasingly similar individuals.
- **Late generations.** Population may be near-homogeneous ("premature convergence"); only mutation and the occasional lucky crossover make progress.

Signs of premature convergence:

- The standard deviation of population fitness drops to near zero early.
- The best-of-generation curve plateaus long before generation G.

Remedies: increase mutation rate, reduce tournament size k, or increase N.

GA vs SA vs LS — benchmark comparison

Experiment. 5 independent trials, $n \in \{40, 60\}$, same random Euclidean instances.

Method	$n = 40$ mean cost	$n = 60$ mean cost
LS first-improvement	524	682
LS best-improvement	521	679
SA ($\alpha = 0.95$, $K = 300$, $\times 1$)	509	665
SA multi-start ($\times 3$)	503	658
GA ($N = 60$, $G = 150$, OX)	511	669
Memetic GA ($N = 50$, $G = 100$, 2-opt)	**498**	**649**

Takeaway. Pure GA and SA are competitive; the **Memetic GA** (GA + 2-opt per offspring) is consistently best. The advantage of population-based methods grows with n.